NEURAVIC

Backend System Architecture Design

Version 1.0 · 18 October 2026 · Status: proposal for review
Source of truth for facts: repository neuravic-co/neuravic @ 2d9a1c2

This document designs the backend that the current Neuravic frontend lacks. Sections 1-2 state what exists (verified in code). Sections 3 onward are **proposals**; where a choice is open it is listed in section 15 and flagged as a decision.

1. Company and product context

Neuravic is a multidisciplinary engineering company in Dubai and Cairo (AI systems, software platforms, product experience, deployment and operations). Its public site presents two systems: **DARKAGENT** (governed multi-agent platform) and **COOLACCESS_DXB** (request/approval/access workflows). The backend serves the company website and the planned client Portal; the two products are treated as separate systems that connect to it.

2. Current state (verified)

Area	Today	Implication for backend
Frontend	Next.js 16 / React 19 via vinext, Cloudflare Workers output, single page	Same-origin API is possible: /api/* routes or a sibling Worker
Data	db/schema.ts empty; Drizzle SQLite/D1 helper getDb(); hosting.json d1=null, r2=null	Bindings must be created and wired; first migration is yours
Contact	mailto:hello@neuravic.com only	Need lead capture, anti-spam, notifications, storage
Portal	Header button shows toast "Ready to connect when you provide the final URL"	No auth or portal service exists
Auth	app/chatgpt-auth.ts reads oai-authenticated-user-* headers (unused)	Platform-specific; do not rely on it in production
Libraries	zod, react-hook-form, drizzle-orm, shadcn/ui (~60 components)	Shared validation and a ready portal UI kit
Quality	Only eslint; no tests, no CI workflow in repo	Add CI, tests and migrations gate

3. Requirements

3.1 Functional

• **FR1 Lead intake**: accept project enquiries from the site (name, email, company, message, optional budget/timeline/service interest), validate, store, notify Neuravic, auto-acknowledge the sender.

• **FR2 Authentication**: email-based login for portal users (magic link or passkey), sessions, logout, invite flow by admin.

• **FR3 Organisations and roles**: client orgs with members; roles: owner, admin, member, viewer; plus internal staff roles.

• **FR4 Portal**: projects per org, service/change requests, approval steps with comments, file attachments, activity feed.

• **FR5 Admin**: lead inbox with status (new, qualified, won, lost), notes, CSV export, user and org management.

• **FR6 Notifications**: email and Slack-compatible webhook on key events.

• **FR7 Content (optional)**: projects, services and team served as JSON so the page stops being hard-coded.

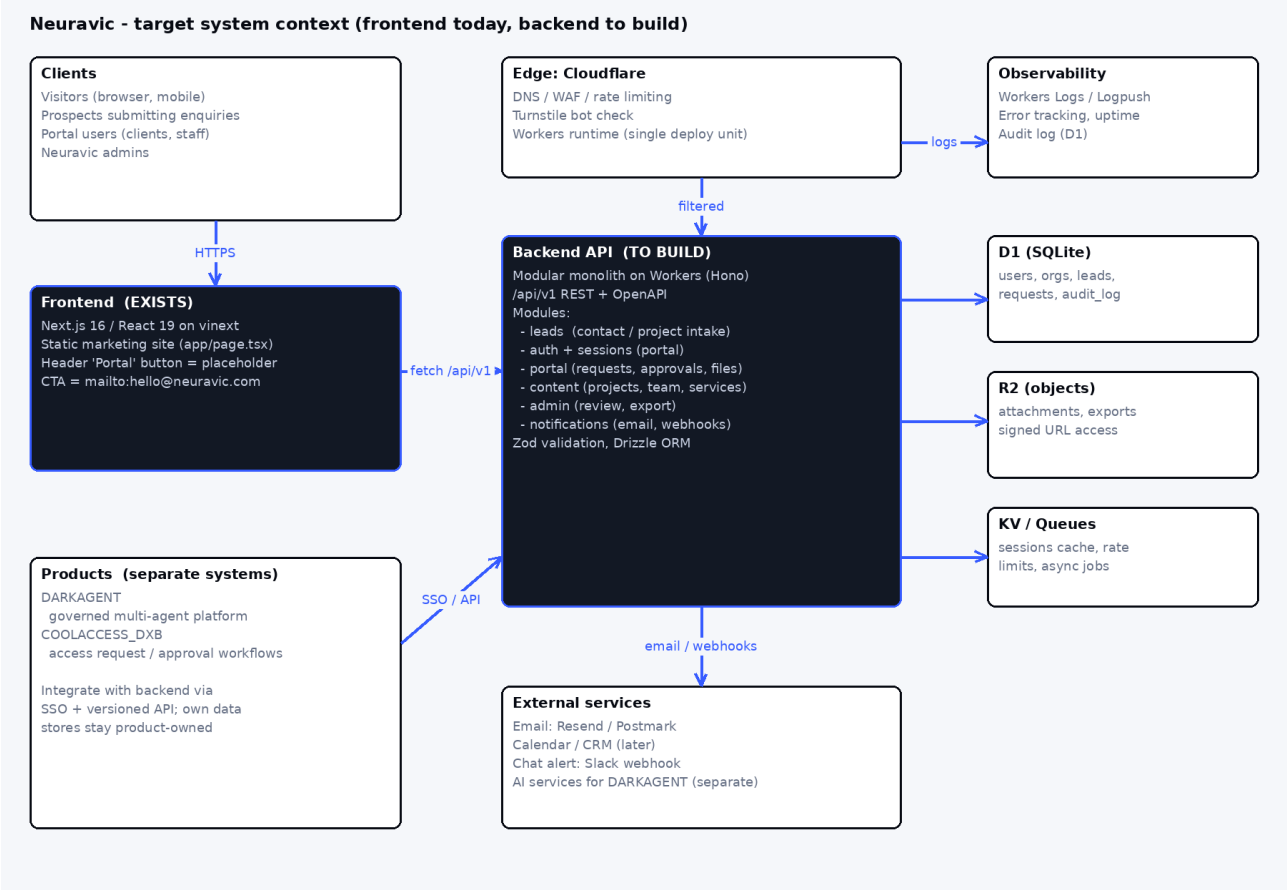
• **FR8 Product bridge**: SSO / token exchange so DARKAGENT and COOLACCESS_DXB can trust Portal identity.

3.2 Non-functional

Attribute	Target (initial, adjust after load data)
Availability	99.9% monthly for lead intake and login
Latency	p95 < 300 ms for reads, < 600 ms for writes at the edge
Security	OWASP ASVS L2 baseline; MFA-capable; least privilege; encrypted at rest and in transit
Privacy	Collect minimum personal data; deletion and export on request; retention schedule; regional requirements (UAE PDPL, Egypt PDPL) to be confirmed with counsel
Scalability	Hundreds of users and thousands of leads/year; design must not need a rewrite for 100x
Operability	One-command deploy, rollback, migrations in CI, structured logs, alerting
Team fit	Small team: one language (TypeScript), one deployable, managed services

4. Architecture overview

Style: **modular monolith** deployed as a Cloudflare Worker (Hono router) next to the existing frontend Worker, with managed storage. Rationale: matches the current build output, removes servers to operate, keeps latency low, and keeps module boundaries explicit so a module can later be extracted.



5. Component design

Module	Responsibility	Owns (tables / storage)	Depends on
edge	WAF, rate limit, Turnstile verify, CORS, request id, security headers	KV rate counters	Cloudflare
leads	Create/list/update leads, spam score, dedupe by email+hash	leads, lead_notes	notifications
auth	Magic link / passkey, sessions, invites, MFA hooks, token exchange for products	users, sessions, auth_tokens, passkeys	notifications, KV
orgs	Organisations, membership, roles, RBAC policy checks	orgs, memberships	auth
portal	Projects, requests, approvals, comments, activity feed	projects, requests, approvals, comments	orgs, files
files	Upload via signed URL, virus-scan hook, download authorisation	files; R2 bucket	orgs
content	Services, projects (showcase), team entries, publish flag	content_items	none
admin	Staff-only views and exports over leads, users, orgs	none (reads others via services)	all
notifications	Email, Slack webhook, templating, retries via Queue	outbox, delivery_log	Queues, email vendor
audit	Append-only record of security and data-changing events	audit_log	all write paths

Rule: modules call each other only through their exported service interface, never another module's tables. Enforce with an import-boundary lint rule (eslint-plugin-boundaries or dependency-cruiser).

5.1 Layering inside each module

```
route (Hono handler)  -> parse + zod validate -> authz policy check
  -> service (business logic, transactions)
  -> repository (Drizzle queries)  -> D1 / R2 / KV
  -> emits domain event -> outbox -> Queue consumer (email, webhook, audit)
```

5.2 Repository layout (proposed)

```
neuravic/
  app/ components/ ...           existing frontend (unchanged)
  api/
    src/index.ts                 Hono app, route mounting, error handler
    src/modules/{leads,auth,orgs,portal,files,content,admin,notifications,audit}/
    src/lib/{env,errors,ids,logger,rbac}.ts
  packages/shared/               zod schemas + TS types used by web and api
  db/schema.ts  drizzle/         migrations (generated by drizzle-kit)
  api/openapi.yaml               generated from zod
  .github/workflows/ci.yml       lint, typecheck, test, migrate, deploy
```

6. Data model (initial)

SQLite/D1 dialect via Drizzle. IDs are ULIDs (text) so rows sort by time and are safe to expose. All timestamps are UTC ISO-8601 text or epoch integers (pick one and keep it). Soft delete via deleted_at where retention requires it.

Table	Key columns	Notes
users	id, email (unique, lowercased), name, status, staff_role?, created_at	Staff and clients in one table

Table	Key columns	Notes
orgs	id, name, slug, created_at	Client company
memberships	user_id, org_id, role (owner|admin|member|viewer)	Composite PK
sessions	id (hash), user_id, expires_at, ip_hash, ua_hash	Server-side; revocable
auth_tokens	id (hash), user_id/email, purpose, expires_at, used_at	Magic link, single use, 15 min
passkeys	id, user_id, public_key, counter, transports	WebAuthn
leads	id, name, email, company, message, service_interest, budget_band, source, status, spam_score, consent_at, created_at	Index on (status, created_at), email
lead_notes	id, lead_id, author_id, body, created_at	
projects	id, org_id, name, status, created_at	Portal project
requests	id, project_id, type, title, body, state, requested_by, created_at	Service or change request
approvals	id, request_id, approver_id, decision, comment, decided_at	Workflow step
comments	id, request_id, author_id, body, created_at	
files	id, org_id, request_id?, r2_key, name, size, mime, sha256, scan_state, uploaded_by	Object in R2, metadata in D1
content_items	id, kind (service|project|team), slug, data (json), published	Optional CMS-lite
outbox	id, topic, payload, state, attempts, next_attempt_at	Transactional outbox
audit_log	id, at, actor_id, action, target_type, target_id, ip_hash, meta (json)	Append-only

Relationship summary: orgs 1-N memberships N-1 users; orgs 1-N projects 1-N requests 1-N approvals/comments/files; leads are standalone until converted (optional converted_org_id).

7. API design

REST over HTTPS, JSON, base path **/api/v1**, errors as RFC 9457 problem+json, cursor pagination (?cursor=&limit=), idempotency via Idempotency-Key header on POST, OpenAPI generated from zod.

Method + path	Auth	Purpose
POST /leads	public + Turnstile	Create lead (FR1). Returns 202 + id
GET /leads, GET /leads/:id, PATCH /leads/:id	staff	Inbox and status changes
POST /auth/magic-link	public, rate-limited	Request login email (always 202, no user enumeration)
POST /auth/verify	token	Exchange token for session cookie
POST /auth/logout, GET /me	session	Session management, current user + memberships
POST /auth/passkeys/*	session	Register / authenticate passkeys
GET/POST /orgs/:id/members	org admin	Invite and manage members
GET/POST /projects, GET /projects/:id	member	Portal projects
GET/POST /projects/:id/requests	member	Create and list requests
POST /requests/:id/approvals	approver	Approve or reject with comment
POST /requests/:id/comments	member	Discuss a request
POST /files/upload-url, GET /files/:id/download-url	member	Short-lived signed R2 URLs
GET /content/{services,projects,team}	public, cached	Optional site content

Method + path	Auth	Purpose
POST /token/exchange	session	Short-lived JWT for DARKAGENT / COOLACCESS_DXB
GET /health, GET /version	public	Liveness and build info

7.1 Lead intake contract (first endpoint to ship)

```
POST /api/v1/leads
{ "name": "string 1..120", "email": "email", "company": "string?",
  "message": "string 10..4000", "serviceInterest": ["ai"|"software"|"product"|"ops"],
  "budgetBand": "string?", "consent": true, "turnstileToken": "string",
  "website": ""  // honeypot, must be empty }

202 { "id": "01J...", "status": "received" }
400 problem+json (validation)   429 rate limited   422 captcha failed
```

Frontend change needed (small): replace the mailto button with a form built from the existing shadcn Form/Input/Textarea + react-hook-form + the shared zod schema, posting to this endpoint; keep the mailto as a fallback link.

8. Authentication and authorisation

• **Method**: passwordless. Primary: email magic link (single use, 15 minutes, hashed in DB). Add passkeys (WebAuthn) in phase 2; offer TOTP for staff.

• **Sessions**: opaque random id in an HttpOnly, Secure, SameSite=Lax cookie; hash stored server-side; 30-day sliding with 12-hour absolute re-auth for staff; revocable.

• **CSRF**: SameSite plus a per-session double-submit token on state-changing requests; strict Origin check.

• **Authorisation**: central policy function can(user, action, resource) with RBAC (role per org) and ownership checks; every route declares its policy; deny by default.

• **Staff vs client**: staff_role on users (admin, delivery, sales) with a separate admin route group and mandatory MFA.

• **Products SSO**: token exchange issues a 5-minute EdDSA JWT (aud = product, claims: sub, org, role); products verify with a JWKS endpoint. Keys rotate via a kid header.

• **Build vs buy**: Auth is the riskiest custom piece. Buying (Clerk, WorkOS, Auth0) is a legitimate alternative; decision D3 in section 15.

9. Security architecture

Threat	Control
Spam and bots on /leads	Turnstile, honeypot field, per-IP and per-email rate limit (KV), content heuristics, spam_score, quarantine state
Injection	Drizzle parameterised queries, zod on every input, output encoding in email templates
Account takeover	Single-use short tokens, no user enumeration, passkeys/MFA, session rotation on privilege change, login alerts
Broken access control	Deny-by-default policy layer, org scoping in every repository query, automated tests per role
Data exposure	Encryption at rest (D1/R2), signed short-lived file URLs, PII hashed where only comparison needed (ip, ua), logs without bodies
Secrets	Wrangler secrets / environment secrets only; none in repo; rotation runbook; secret scanning in CI

Threat	Control
Supply chain	Lockfile, npm audit and Dependabot, pinned CI actions, minimal dependencies in the Worker
Abuse of uploads	Size and MIME allow-list, sha256, scan hook, download as attachment, separate bucket domain
Headers	CSP, HSTS, X-Content-Type-Options, Referrer-Policy, frame-ancestors none
Auditability	Append-only audit_log for auth, role, export and delete events

Privacy: store consent_at for leads, define retention (for example, unqualified leads deleted after 12 months), provide delete and export for personal data. Confirm legal basis and residency obligations with counsel before launch.

10. Cross-cutting concerns

• **Async work**: domain events are written to the outbox in the same transaction, then drained to Cloudflare Queues; consumers send email/webhooks with retries and a dead-letter queue.

• **Caching**: public content endpoints use Cache-Control and edge cache; authenticated responses are never cached at the edge.

• **Error handling**: typed AppError to problem+json mapping; unknown errors return a request id, details go to logs.

• **Observability**: JSON logs with request id, user id (hashed), module, latency; Workers Logs / Logpush; uptime probe on /health; alerts on 5xx rate, queue backlog, auth failures spike.

• **Configuration**: typed env module validated at start-up (zod); separate local, preview and production environments with separate D1/R2/KV.

11. Deployment and environments

Environment	Purpose	Data
local	wrangler dev with --local --persist-to (already used by npm start)	Local SQLite, seeded fixtures
preview	One per pull request via Cloudflare/Vercel preview or wrangler versions	Ephemeral D1 with migrations applied
staging	Mirror of production, used for QA and demos	Anonymised data only
production	neuravic.com, api route /api/*	Real data; backups on

```
push -> CI: install, lint, typecheck, unit + integration tests (Miniflare/D1),
  drizzle-kit check, build, OpenAPI diff, secret scan
merge to main -> deploy staging -> smoke tests -> manual gate -> migrations -> deploy production
rollback: wrangler rollback (code) ; forward-only migrations with expand/contract
```

Backups: D1 Time Travel for point-in-time restore plus a nightly export to R2 in a separate account/bucket; restore drill each quarter. RPO 24 h or better, RTO 4 h.

Fallback hosting (decision D1): if Cloudflare is not chosen, the same modules map to a Node/TypeScript service (container) + PostgreSQL + S3-compatible storage + a queue; Drizzle supports Postgres and the zod/OpenAPI contract is unchanged.

12. Testing strategy

• **Unit**: services and policy function (Vitest), target 80% on auth, rbac, leads.

• **Integration**: Hono app against Miniflare D1/R2/KV, one test per endpoint and role.

• **Contract**: OpenAPI diff in CI blocks breaking changes; frontend generates types from the spec or the shared zod package.

• **E2E**: Playwright (browsers already available) for lead form submission and login flow on preview.

• **Security**: dependency audit, secret scan, authorisation matrix tests, periodic external pen test before opening the Portal to clients.

13. Delivery roadmap

Phase	Scope	Exit criteria
0 Foundations (1 wk)	api/ skeleton, env, CI, D1 + R2 + KV bindings in hosting config, first migration, logging, /health	CI green, deploy to staging
1 Lead intake (1-2 wk)	FR1 + notifications + Turnstile + admin list endpoint; frontend form	Real enquiry creates a row, email arrives, spam blocked
2 Auth and orgs (2-3 wk)	Magic link, sessions, orgs, roles, invites, audit log	Role matrix tests pass; Portal button opens login
3 Portal core (3-5 wk)	Projects, requests, approvals, comments, files, activity feed	One pilot client runs a real workflow
4 Admin and content (2 wk)	Admin console, exports, content API, retention jobs	Staff manage without DB access
5 Product bridge (2 wk)	Token exchange + JWKS, SSO for DARKAGENT and COOLACCESS_DXB	Single sign-on across products

Estimates assume one backend engineer with frontend support; they are planning figures, not commitments.

14. Risks

Risk	Likelihood	Impact	Mitigation
Custom auth flaws	Medium	High	Use vetted libraries or buy; pen test; staff MFA
D1 limits (single-writer SQLite, size caps) as portal grows	Low-Med	Medium	Repository layer isolates SQL; Postgres migration path documented
Scope creep into building products' features	Medium	Medium	Products own their data; bridge via SSO and API only
Lead email deliverability	Medium	Medium	SPF, DKIM, DMARC on sending domain; outbox retries
Regulatory / residency surprises	Low-Med	High	Early counsel review; region and retention decisions in phase 0
Platform lock-in (Cloudflare)	Medium	Low-Med	Hexagonal boundaries, standard HTTP/SQL, OpenAPI

15. Decisions needed from you

ID	Decision	Recommendation
D1	Backend platform	Cloudflare Workers + Hono + D1/R2/KV (fits existing build); alternative Node + Postgres
D2	Portal audience	Define clients vs staff vs both; recommended: both, with orgs
D3	Auth build vs buy	Build magic link + passkeys if scope stays small; buy if SSO/SAML for enterprise clients is expected

ID	Decision	Recommendation
D4	Email provider	Resend or Postmark on a neuravic.com sending subdomain
D5	Product integration depth	SSO only first; shared data later if needed
D6	Data residency and retention	Confirm with counsel before phase 2
D7	Repo layout	Monorepo (api/ + packages/shared/) as in 5.2

Appendix A. ADR summary

ADR	Decision	Consequence
1	Modular monolith	Single deploy, simple ops; discipline needed to keep boundaries
2	D1 + Drizzle, repository pattern	Cheap, local-first; swap to Postgres without touching services
3	Passwordless auth, opaque server sessions	Easy revocation; needs session store reads
4	Transactional outbox + Queues	No lost emails/events; slightly more code
5	OpenAPI from zod, shared package	One schema for form and API; prevents drift